

1) Mecanismos de Protección de acceso al Segmento en Modo Protegido:

- a. Realice un diagrama completo que permita comprender el mecanismo de protección de memoria del 80x386 que permite un no acceso directo al segmento. Explique detalladamente como interpreta un valor en un registro de segmento si TI=1 o TI=0.
- b. ¿Cuáles son las condiciones que deben cumplirse para que el selector correspondiente pueda acceder al segmento? ¿Cuál es la relación entre RPL y DPL?
- c. Explique que son y para qué sirven los siguientes registros y bits: GDTR, LDTR, RPL, bit G, bit D, bit P, bit S, bit E, bit A.
- d. Práctica: Ejercicio de MP (Interpretación y funcionamiento) Consigna: Realice un diagrama interpretando en Modo Protegido el siguiente selector: CS=A3FF.

2) Descriptores de Segmento y Descriptores de Sistema:

- a. ¿Cuántas tablas de descriptores existen en la arquitectura 80x386? Explique cada una de ellas. NOTA: No es lo mismo “tablas de descriptores” que “descriptores”.
- b. Explique de qué forma actúa el bit S como atributo de un descriptor. Utilice esta información para decir en qué tipo de descriptor se convierte el mismo. ¿Qué “tipos” de estos descriptores existen?
- c. Explique el concepto y funcionamiento de un “Call via Gate”.

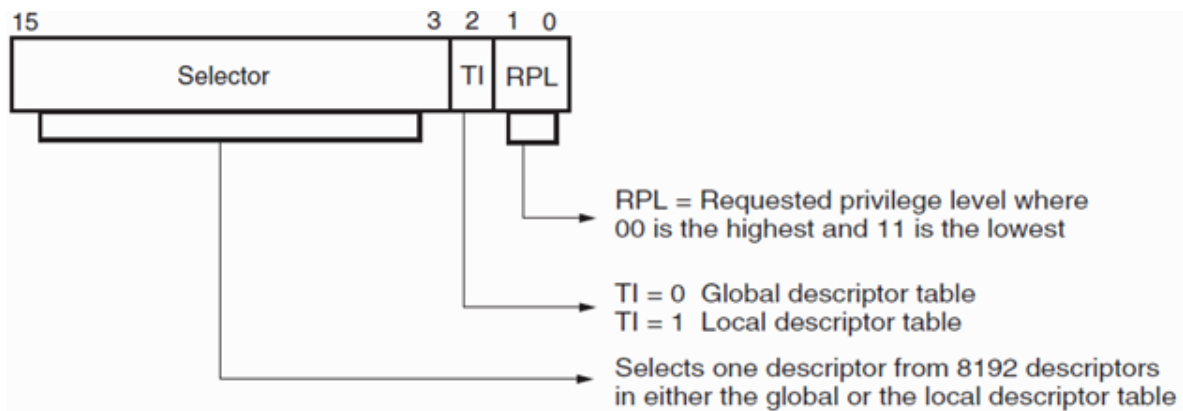
3) Segmento de Estado de la Tarea (TSS):

- a. ¿Qué se busca con el uso del TSS relacionado al uso del procesador y la conmutación de tareas?
- b. Explique el funcionamiento y realice un diagrama de interpretación del mismo. Esquema del TSS.

4) Paginado:

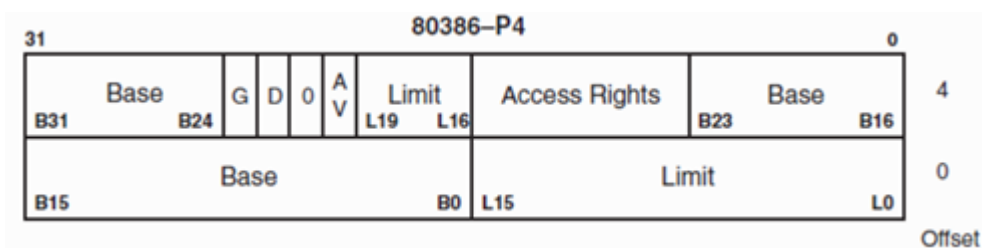
- a. Explique conceptualmente para qué sirve el proceso de paginado y como se relaciona con el sistema de segmentado.
- b. ¿Qué hace la unidad de paginado? ¿Cómo lo hace?

1-a) La tarea tiene un nivel de privilegio, genera un selector para acceder a una dirección dentro de un segmento con su propio nivel de privilegio. El selector tiene la siguiente estructura

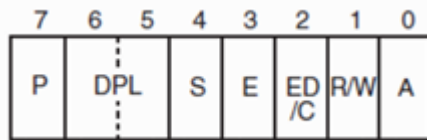


El índice se compone de 13 bits, que permite identificar uno de los 8192 descriptors. El bit TI indica si se trata de una tabla de descriptors global o local. Los 2 bits de RPL definen el privilegio.

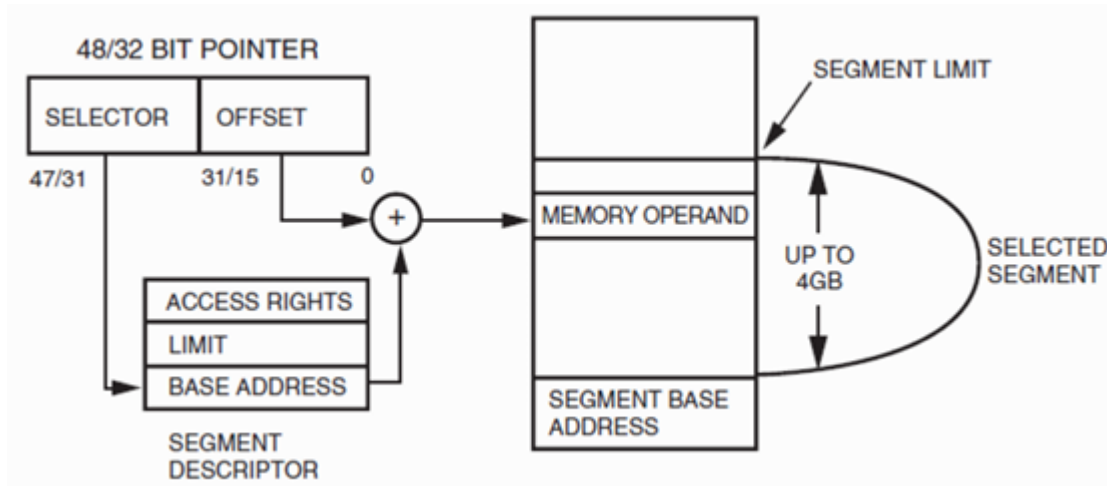
Tras seleccionar al descriptor correspondiente en la tabla de descriptors (GDT o LDT, en función de TI), se compara el nivel de prioridad del descriptor (DPL) con el del selector (RPL), pudiendo acceder si $RPL \geq DPL$ (siendo 00 el de mayor prioridad, y 11 el de menor). Si el descriptor es de código, el bit C del mismo define si se respeta el nivel de privilegio (C=1) o si se omite. El descriptor del 80386 se describe a continuación



Donde el byte de derechos de acceso es:



El sistema de acceso en modo protegido se representa mediante el siguiente diagrama:



1-b) Para que se puede acceder, $RPL \geq DPL$ donde el mayor nivel de prioridad lo representa 00 y disminuye al mínimo que es 11.

Si no se cumple, no se puede acceder al descriptor salvo que sea mediante un call via gate. Hay que tener en cuenta que el privilegio mayor, numéricamente es menor ejemplo (PL se compone de dos bits por eso va del 0 al 4):

PL = 0: Kernel (parte del sistema operativo). Mayor nivel de privilegio (PL)

PL = 1: Servicios del sistema (parte del SO).

PL = 2: Extensiones del sistema operativo.

PL = 3: Aplicaciones. Menor nivel de privilegio (PL)

1-c)

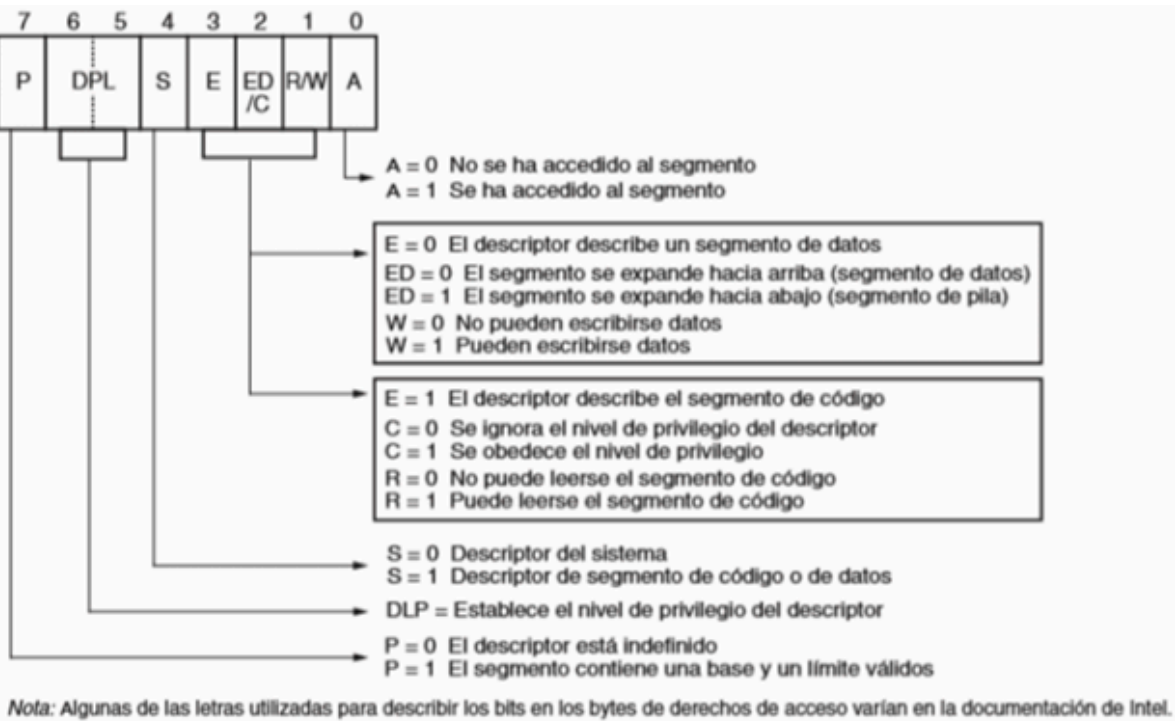
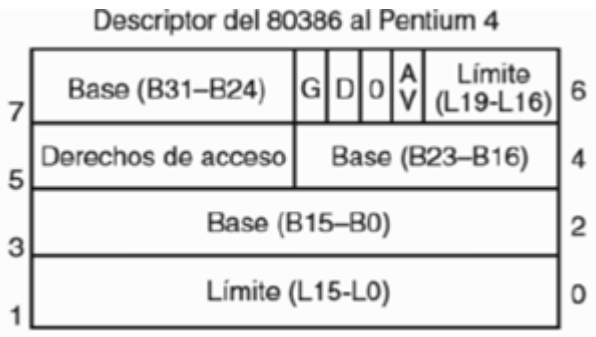
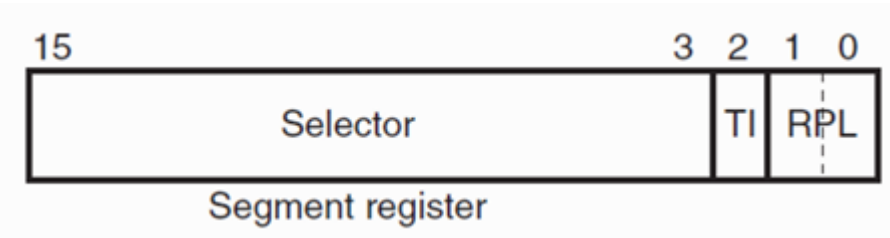
LGDT(Load Global Descriptor Table): Instrucción que carga la base y el límite de la tabla global de descriptores (GDT) en el registro GDTR.

LGDT GDTR;Inicializa GDTR

GDTR (Global Descriptor Table Register): Registro que contiene la dirección base del descriptor y su límite. Al ser descriptor global, TI=0 (TI:Table Indicator).

RPL (Requested Privilege Level): Nivel de acceso requerido por un segmento de memoria.

LDTR (Local Descriptor Table Register): Registro que contiene la dirección base del descriptor y su límite. Al ser descriptor local, TI=1 (TI:Table Indicator).



- G: Bit de granularidad. Si G=0 se trata de un segmento de 1MB. Si G=1 el tamaño del segmento se multiplica por 4KB
- D: Si D=0 opera con instrucciones de 16bits. Si D=1, emplea instrucciones de 32bits.
- P: Indica si el descriptor está definido (P=1) o no (P=0).
- S: Indica si se trata de un descriptor de sistema (S=0) o de código/datos (S=1).
- E: Indica si es segmento de datos (E=0) o de código (E=1).
- A: Indica si se ha accedido al segmento (A=1) o no (A=0).

1-d)

CS=A3FF

1010 0011 1111 11**11**

RPL:11

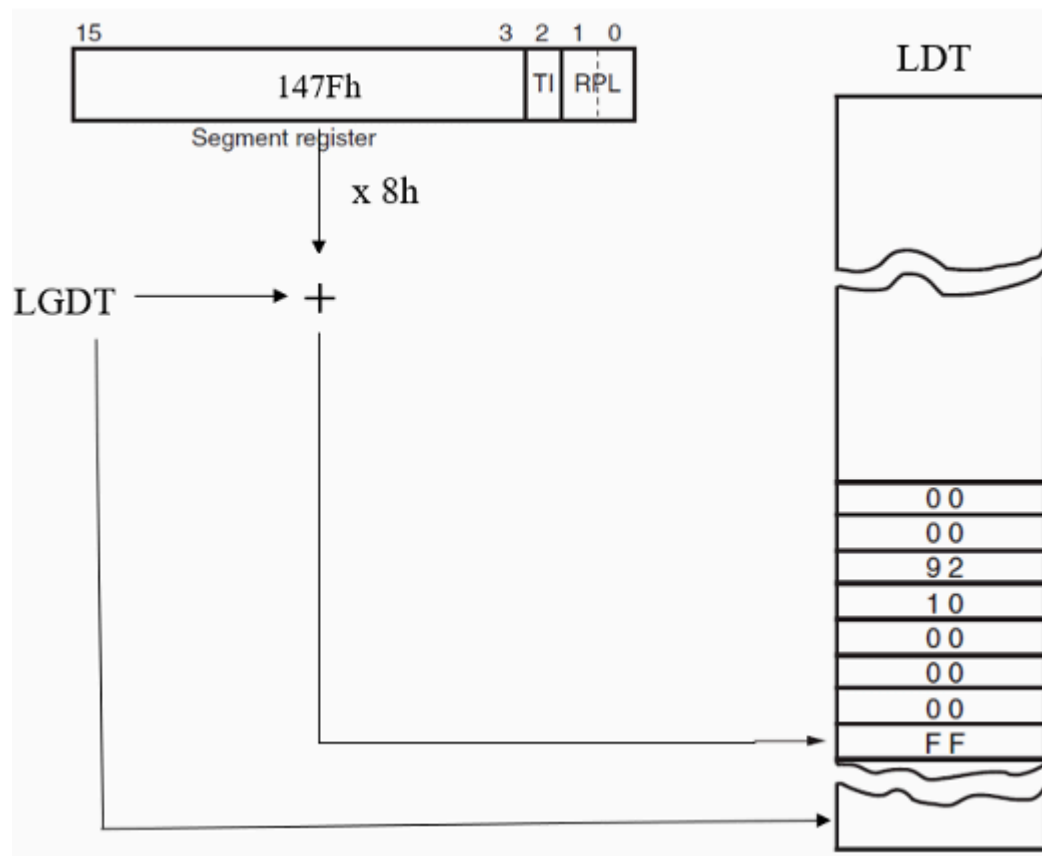
TI:1 Descriptor local

Índice: 0001 0100 0111 1111

1 4 7 F

El selector selecciona uno de los 8912 descriptors de la LDT, en la dirección dada por la suma de la dirección base y el desplazamiento (el índice multiplicado por 8h):

LGDT +147Fh * 8h.



2-a) Existen tres tipos de tablas de descriptores:

- GDT: Tabla global de descriptores contiene 8192 descriptores de 8 bytes, su dirección de inicio se define en el registro GDTR y es única para todo el sistema.
- LDT: Tabla local de descriptores, de cada tarea en particular. Contiene 8192 descriptores de 8 bytes y la dirección de inicio se define en el LDTR.
- IDT: Tabla de descriptores que contiene los vectores de interrupción. Se accede con la dirección base y el límite almacenado en IDTR.

2-b) El bit "S" indica si se trata de un descriptor de sistema (S=0) o de código o datos (S=1)

Dentro de los descriptores de sistema se encuentran:

- DeLDT(tipo=2)
- DeGate(tipo=4 a 7)
- DeTSS(tipo=1, 3, 9)

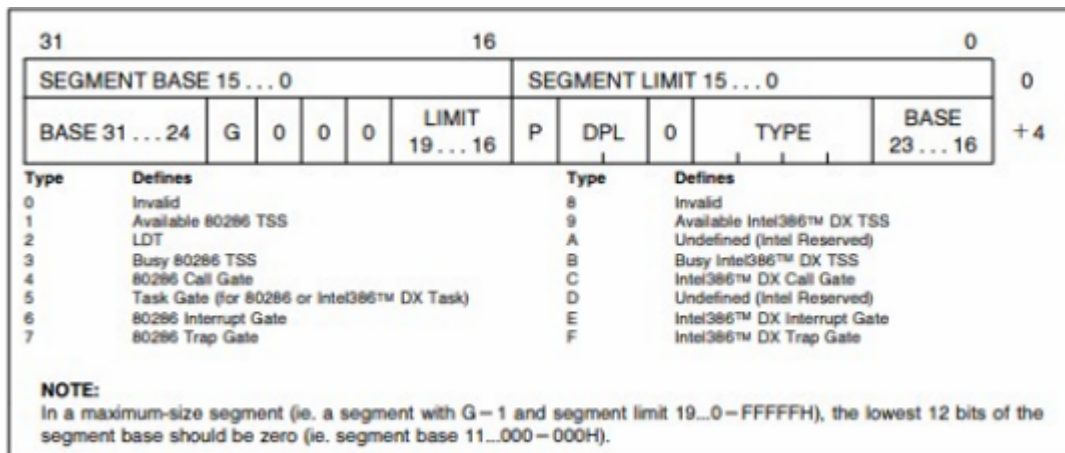


Figure 4-7. System Segments Descriptors

Los descriptores de segmento son:

- De código
- De datos

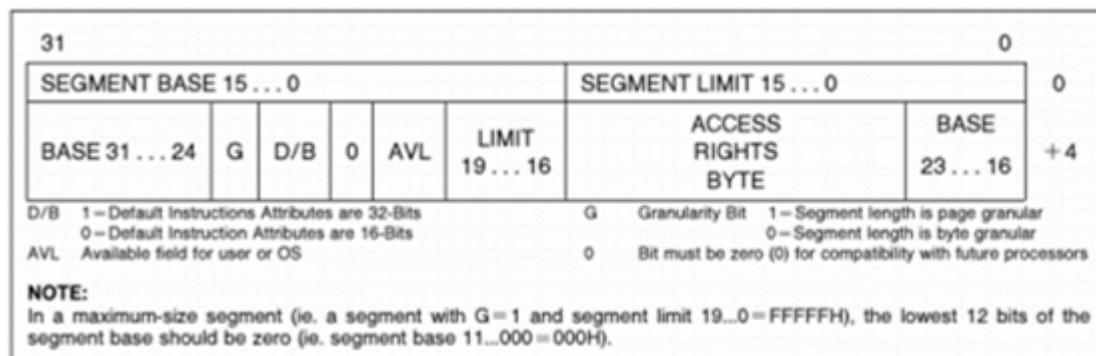


Figure 4-6. Segment Descriptors

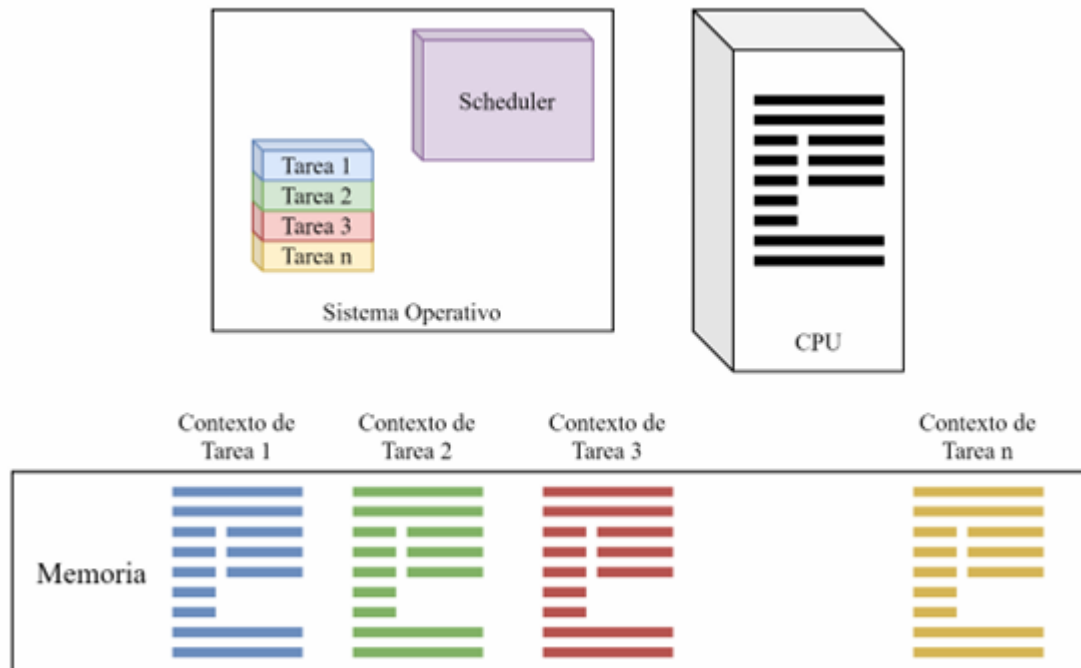
2-c) Un “Call via Gate” permite que una tarea con menor nivel de privilegio pueda acceder a rutinas de mayor nivel, modificando momentáneamente los niveles de prioridad para permitirlo. Este tipo de mecanismo es esencial en sistemas operativos modernos que utilizan protección de memoria porque permite que aplicaciones de usuario usen función del Kernel y system calls de una forma que puedan ser controladas por el sistema operativo.

Las “Call Gates” usan un selector de valor especial para referenciar a un descriptor accedido

por el GDT o el LDT, que contiene la información necesaria para la llamada a través de niveles de privilegio. Esto es similar al mecanismo usado por las interrupciones. Si esto se realiza a través de un CALL indica que la rutina llamada deberá retornar a la tarea que la ejecuta, por lo que se producirá un anidamiento de tareas (nest).

3-a) Permitir la ejecución de múltiples tareas en “simultaneo”, simulando multiples procesadores. Lo que ocurre es que el procesador conmuta entre tareas a tal velocidad, de forma que el usuario percibe que se ejecutan en simultaneo. Que tarea se ejecuta y en qué momento, es determinado por el sistema operativo.

3-b) Las distintas tareas presentan un nivel de prioridad, y son conmutadas por medio del scheduler a intervalos regulares de tiempo (ticks). Cuando una tarea deja de ejecutarse, se guarda su contexto de ejecución y se procede a cargar el de la próxima tarea a ejecutar. Se ejecuta dicha tarea, y al finalizar se repite el procedimiento para pasar a la siguiente tarea. De ser necesario, una tarea puede ejecutarse continuamente durante un número mayor de ticks, en función de su estado y prioridad.



4-a) La paginación es una estrategia de organización de la memoria física que consiste en dividir la memoria en porciones de igual tamaño. A dichas porciones se las conoce como páginas físicas o marcos. La división de la memoria en páginas facilita la gestión de la memoria física. La unidad de manejo de memoria (MMU) consiste en una unidad de segmentación (similar a la del 80286) y una unidad de paginado (nuevo en este microprocesador). La segmentación permite el manejo del espacio de direcciones lógicas agregando un componente de direccionamiento extra, que permite que el código y los datos se puedan reubicar fácilmente. El mecanismo de paginado opera por debajo y es transparente al proceso de segmentación, para permitir el manejo del espacio de direcciones físicas. Cada segmento se divide en uno o más páginas de 4 kilobytes. Para implementar un sistema de memoria virtual (aquél donde el programa tiene un tamaño mayor que la memoria física y debe cargarse por partes (páginas) desde el disco rígido), el 80386 permite seguir ejecutando los programas después de haberse detectado fallos de segmentos o de páginas. Si una página determinada no se encuentra en memoria, el 80386 se lo indica al sistema operativo mediante la excepción 14, luego éste carga dicha página desde el disco y finalmente puede seguir ejecutando el programa, como si hubiera estado dicha página todo el tiempo. Como se puede observar, este proceso es transparente para la aplicación, por lo que el programador no debe preocuparse por cargar partes del código desde el disco ya que esto lo hace el sistema operativo con la ayuda del microprocesador. La memoria se organiza en uno o más segmentos de longitud variable, con tamaño máximo de 4 gigabytes. Estos segmentos, como se vio en la explicación del 80286, tienen atributos asociados, que incluyen su ubicación, tamaño, tipo (pila, código o datos) y características de protección. La unidad de segmentación provee cuatro niveles de protección para aislar y proteger aplicaciones y el sistema operativo. Este tipo de protección por hardware permite el diseño de sistemas con un alto grado de integridad. El 80386 tiene

dos modos de operación: modo de direccionamiento real (modo real), y modo de direccionamiento virtual protegido (modo protegido). En modo real el 80386 opera como un 8086 muy rápido, con extensiones de 32 bits si se desea. El modo real se requiere primariamente para preparar el procesador para que opere en modo protegido. El modo protegido provee el acceso al sofisticado manejo de memoria y paginado. Dentro del modo protegido, el software puede realizar un cambio de tarea para entrar en tareas en modo 8086 virtual (V86 mode) (esto es nuevo con este microprocesador). Cada una de estas tareas se comporta como si fuera un 8086 el que lo está ejecutando, lo que permite ejecutar software de 8086 (un programa de aplicación o un sistema operativo). Las tareas en modo 8086 virtual pueden aislarse entre sí y del sistema operativo (que debe utilizar instrucciones del 80386), mediante el uso del paginado y el mapa de bits de permiso de entrada/salida (I/O Permission Bitmap). Finalmente, para facilitar diseños de hardware de alto rendimiento, la interfaz con el bus del 80386 ofrece pipelining de direcciones, tamaño dinámico del ancho del bus de datos (puede tener 16 ó 32 bits según se desee en un determinado ciclo de bus) y señales de habilitación de bytes por cada byte del bus de datos. Hay más información sobre esto en la sección de hardware del 80386.

4-b) La unidad de paginado se ubica entre la unidad de segmentado y la memoria física. Una vez que se la habilita, se encarga de traducir las direcciones lineales en direcciones dentro de las páginas (virtuales).

A partir de la dirección generada por la unidad de segmentado, los primeros 12 bits determinan el offset dentro de la página, los siguientes 12 bits la dirección de entrada a la tabla de páginas y los últimos 10 bits la entrada en el directorio de tablas de paginación (cuyo tamaño es 4K).

Cada entrada en el directorio y en la tabla ocupa 4 bytes. Se multiplica dicha entrada por 4 y se obtiene la dirección de entrada. La entrada tiene el número de página en la cual está la tabla y los atributos.

Tras acceder a la página donde se encuentra la tabla, se repite el procedimiento, pero para el número de entrada en la tabla. Se obtiene la página buscada, y a su dirección de segmento se le suma el offset.